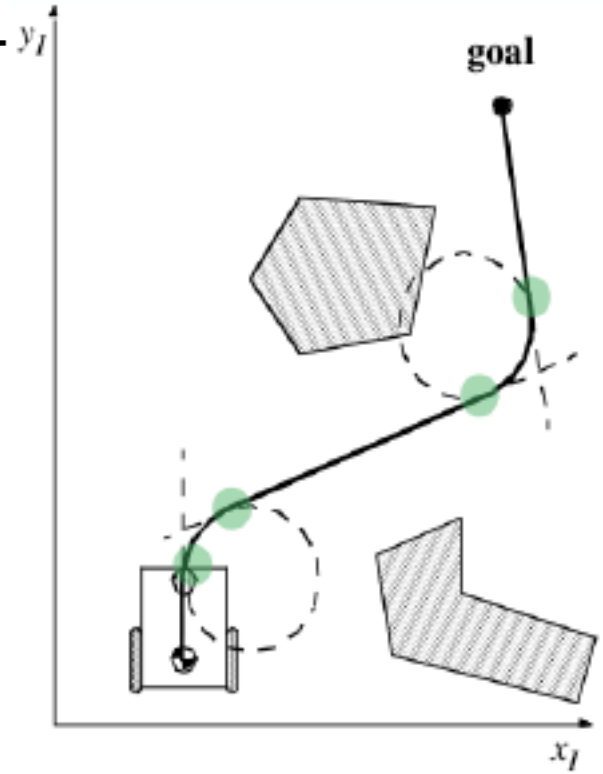
Controls

Prof. Muthukumar

Quick Recap

- Forward Kinematics
  - Pose Prediction given controls ($\boldsymbol{\nu}$,$\boldsymbol{\omega}$)
- Inverse Kinematics
  - Controls ($\boldsymbol{\nu}$,$\boldsymbol{\omega}$) for Pose Regulation
- Path Planning
  - Geometry – path following
- Trajectory Planning
  - Trajectory following (kinematics, time)



- SIMPLIFIED INVERSE KINEMATICS APPROACH
  - Solve the problem by decomposing the trajectory in primitive motion segments
    - Straight lines
    - Segments of a circle or rotation in place

Control

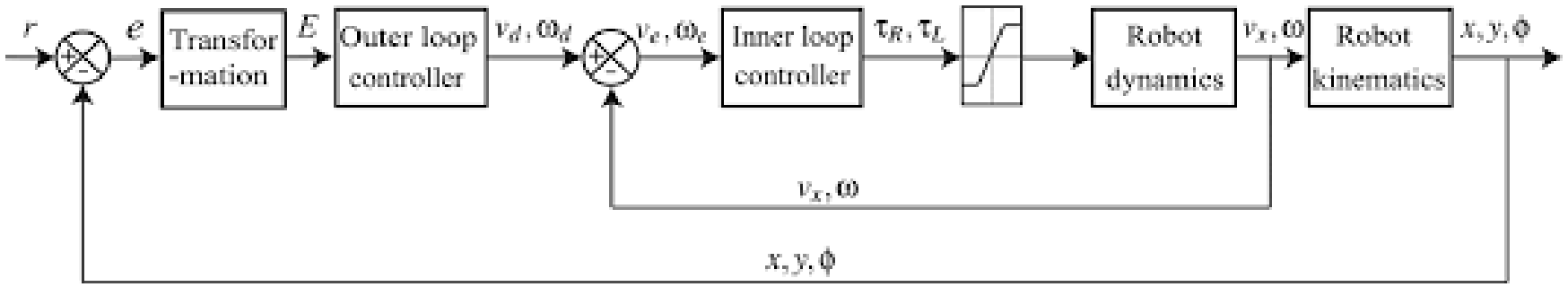
- Now we know how to get there? (We'll visit this problem in detail later)
- Can you follow directions to get there?
- Can you stick to the path to get there?

- Errors always exist no matter how good you configure your robot.
- Two DC motors are required to run in the same speed to make sure the robot moves in a straight line.
- Given the surface and motor variations – speed and rotational errors always exists.

- Solution:
  - Control the robot to get there by following directions

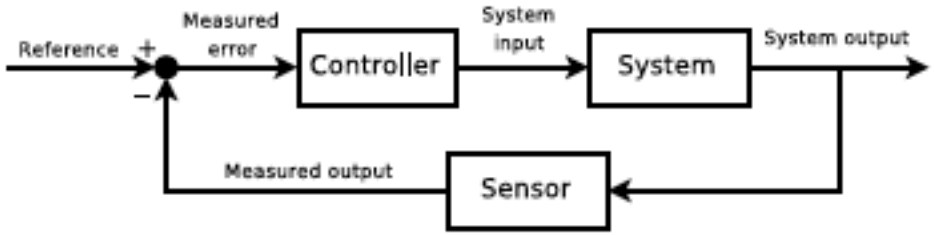
Why need Control?

- Two control mechanisms in mobile robots
  - PID speed control of the motors (individually)
    - Speeds are maintained as constant as possible irrespective to external variations.
  - PID for trajectory or pose control (angle)
  - Cascaded Controllers (collectively)
    - two PID controllers are used conjointly to yield a better control response
    - outer loop controller for velocity and twist, while the inner loop controller controls torque, force, etc.



Control theory

- Control theory is an interdisciplinary branch of engineering and mathematics that deals with the behavior of dynamical systems with inputs, and how their behavior is modified by feedback.



- Topics studied in control theory are stability (whether the output will converge to the reference value or oscillate about it), controllability and observability.

State of the System

- A system is linear when the output is a sum of products between variables and constant coefficients.
- It is time invariant when the same inputs always yield the same output.
- model a controller as being linear and time–invariant (LTI)
- LTI controllers are represented with equations in a standardized format called State Space Form.
- state of the system, X, is usually a vector containing the physical position of the system (robot) and the derivative with respect to time for each dimension of the controller's position.

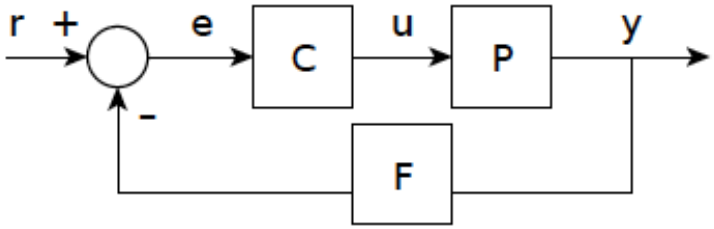
$$\dot{X} = [\dot{x}\ \dot{y}\ \dot{\varphi}\ \dot{\phi}_R\ \dot{\phi}_L]^{\mathrm{T}}$$

- Or Four-State Error Model of the System $\dot{X} = [\dot{x}\ \dot{y}\ \dot{s}\ \dot{c}\ ]^{\mathrm{T}}$

Type of controllers and systems

- open-loop controller – fixed input, there is no feedback; no measurement of the system output.
- closed-loop control system - data from a sensor monitoring the system output, controller continuously adjusts input based on reference/error, minimize error to zero.
- Linear versus nonlinear control theory
  - Linear - systems that obey the superposition principle - output is proportional to the input, amenable to powerful frequency domain mathematical techniques (L,Z,F …)
  - Nonlinear - systems are often governed by nonlinear differential equations, often analyzed using numerical methods.
- near a stable point are of interest, nonlinear systems can often be linearized by approximating them by a linear system using perturbation theory, and linear techniques can be used.

Closed-loop transfer function



- The output of the system y(t) is fed back through a sensor measurement F to the reference value r(t). The controller C then takes the error e (difference) between the reference and the output to change the inputs u to the system under control P.
- Using Laplace-transform we can express the above system as:
- Y (s) = P(s)U(s), U(s) = C(s)E(s), and E(s) = R(s) - F(s)Y(s)

$$Y(s) = \left(\frac{P(s)C(s)}{1 + F(s)P(s)C(s)} \right) R(s) = H(s)R(s)$$

- H(s) is the closed loop transfer function. Relation b/w input and output of the system.
- If the poles (denominator) on the function are negative, then the system is stable (output under control).

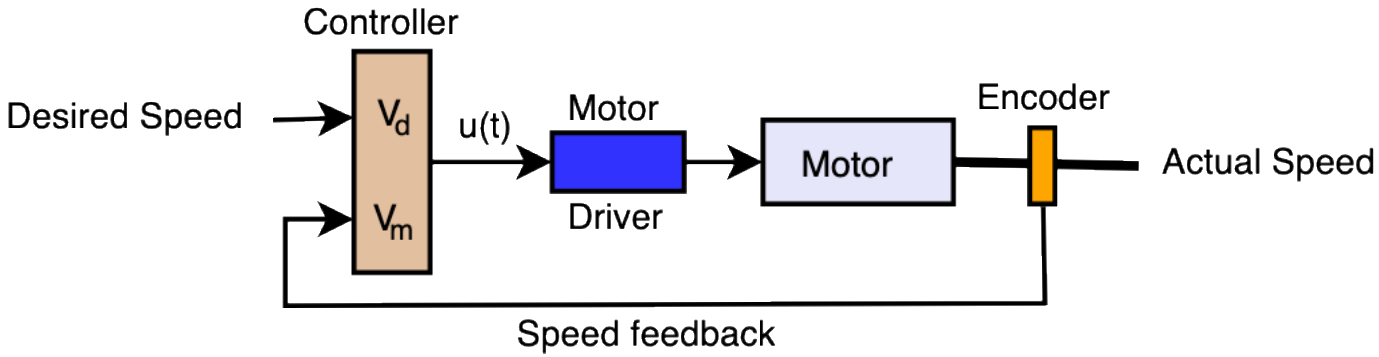
Stability, Controllability and Observability

- A linear system is called bounded-input bounded-output (BIBO) stable if its output will stay bounded for any bounded input.
- Stability for nonlinear systems that take an input is input-to-state stability (ISS), which combines Lyapunov[1] stability and a notion similar to BIBO stability.
- Controllability is related to the possibility of forcing the system into a particular state by using an appropriate control signal.
- Observability is related to the possibility of observing through output measurements the state of a system.

[1] **Lyapunov functions** are scalar functions that may be used to prove the stability of an equilibrium of an ODE.

Motor Speed - Feedback Controller

- Encoder returns angular velocity (rpm)
- controller block is the part that takes the two signals, desired speed and actual speed, and decides what command to give to the motor driver.

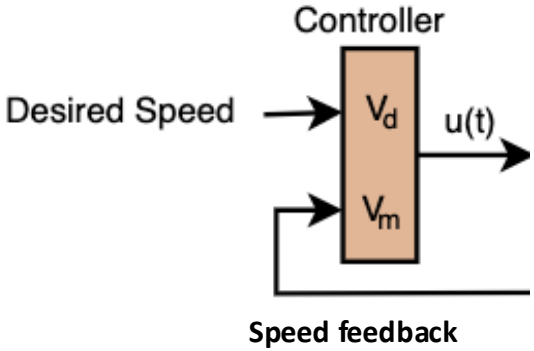


- Bang-Bang (On-off) Control
- Proportional control with feedback
- Proportional Integral Differential (PID) Controller

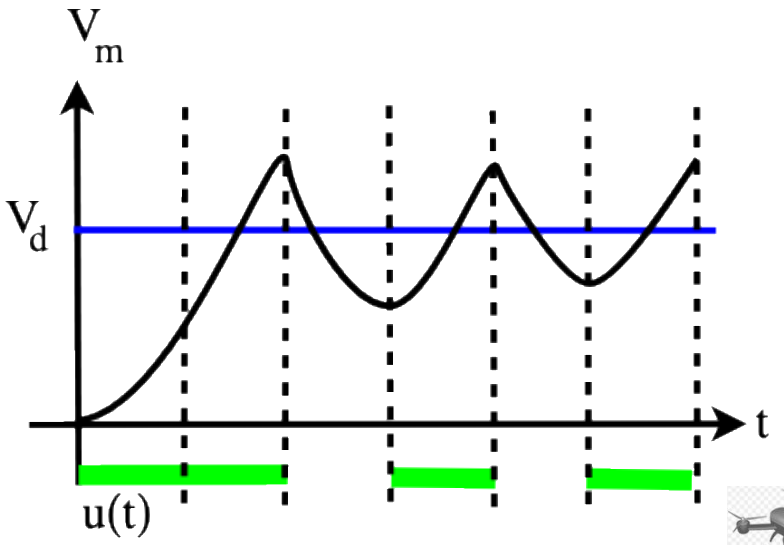
Bang-Bang Control

- Bang-Bang (On-off) Control - A control approach that turns the actuator or motor completely on or off without using proportional values.

$$u(t) = \begin{cases} u_c & \sim\text{if}\sim v_m(t) < v_d \\ 0 & \sim\text{if}\sim v_m(t) \geq v_d \end{cases}$$



- u_c is the control input to the motor driver
- You'll one for each motor
- Encoder values should be translated to φ_R and φ_L
- Issues: oscillation, overrun.

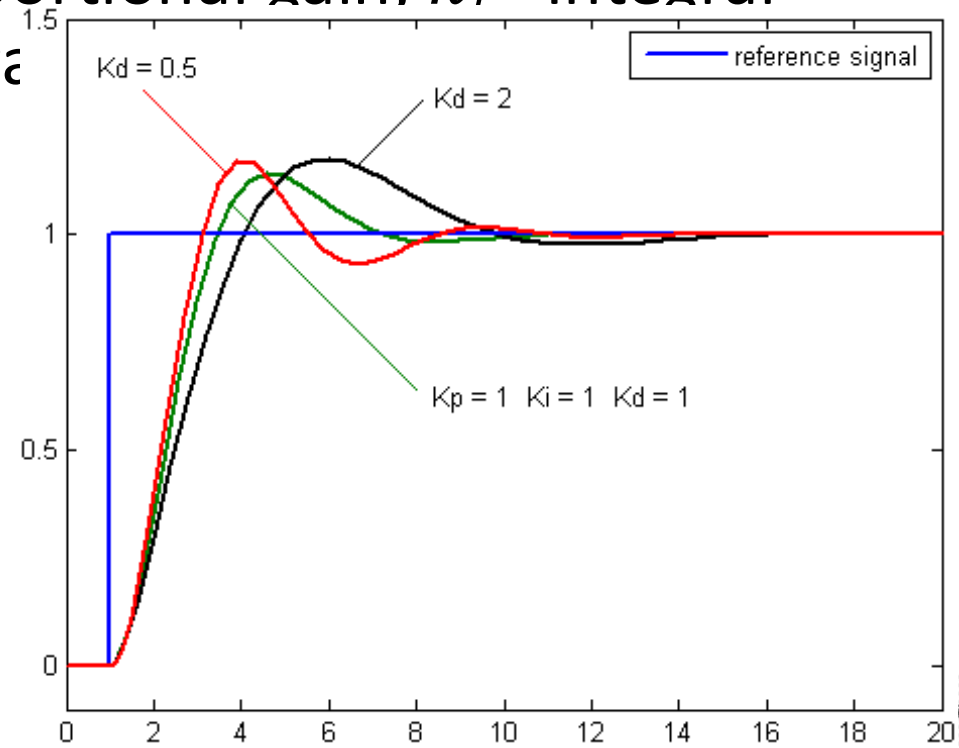


PID Controller

- More robust control term $u(t)$, addresses oscillations, overshoot, and instability.

$$u(t) = k_P e(t) + k_D \frac{de(t)}{dx} + k_I \int_0^t e(t)dt$$

- $e(t)$ – error, k_P - Proportional gain, k_I - Integral gain, k_D - Derivative ga



PID

- PID control approach to control the wheel velocities

$$\dot{\varphi}_R = \frac{1}{\mathrm{r}}(lu(t) + v_c)$$

$$\dot{\varphi}_L = \frac{1}{\mathrm{r}}(-lu(t) + v_c)$$

discrete form of the PID control:

$$U_n = k_P e_n + k_I \Delta t \sum_{i=1}^{n} \frac{e_i + e_{i-1}}{2} + k_D \frac{e_n - e_{n-1}}{\Delta t}$$

$$U_n - U_{n-1} = k_P(e_n - e_{n-1}) + k_I \Delta t \left(\frac{e_n + e_{n-1}}{2}\right) + k_D \frac{e_n - 2e_{n-1} + e_{n-2}}{\Delta t}$$

$$U_n = U_{n-1} + K_P(e_n - e_{n-1}) + K_I(e_n + e_{n-1}) + K_D(e_n - 2e_{n-1} + e_{n-2})$$

where $K_P = k_p$, $K_I = k_I \Delta t/2$, $K_D = k_D/\Delta t$.

Heading to Goal (Control)

- From inverse kinematics (alternate form)

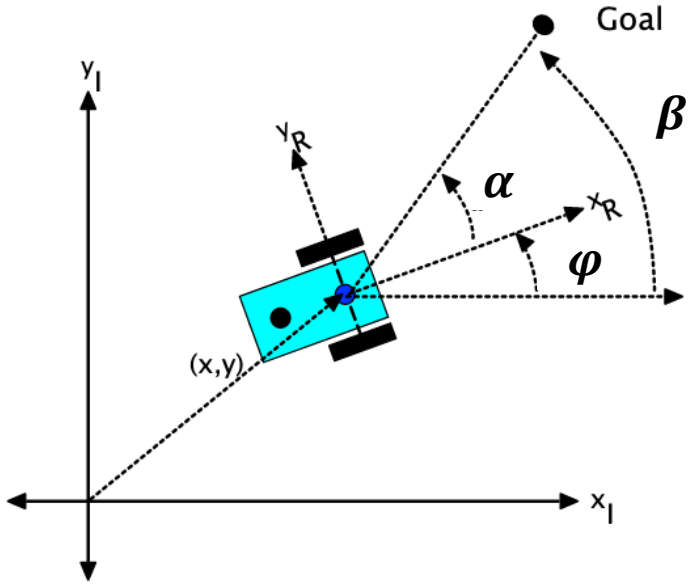
$$v = \frac{r}{2}(\dot{\varphi}_R + \dot{\varphi}_L)$$

$$\omega = \dot{\theta} = \frac{r}{2l}(\dot{\varphi}_R - \dot{\varphi}_L)$$

$$\dot{\varphi}_R = \frac{\mathrm{v}}{\mathrm{r}}(\kappa l + 1)$$

$$\dot{\varphi}_L = \frac{\mathrm{v}}{\mathrm{r}}(-\kappa l + 1)$$

$$\kappa = \frac{\dot{x}\ddot{y} - \dot{y}\ddot{x}}{v^3} = \frac{\dot{\theta}}{v}$$



- Orientation error is $\boldsymbol{\alpha}$
  - Angle to the goal
- Control heading to the goal
  - Find wheel velocities such that v is constant, v_c and $\alpha \rightarrow 0$.

$$\dot{\varphi}_R = \frac{1}{\mathrm{r}}(lk\alpha + v_c)$$

$$\dot{\varphi}_L = \frac{1}{\mathrm{r}}(-lk\alpha + v_c)$$

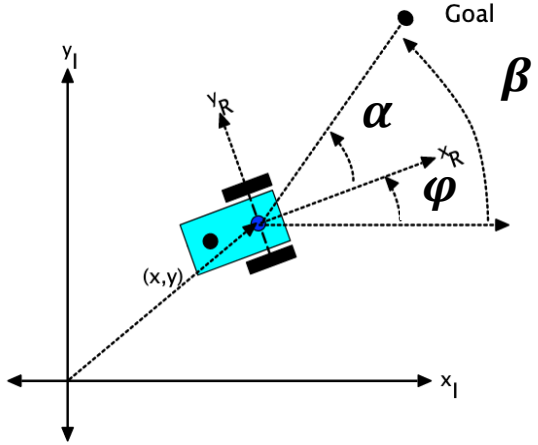
k is a constant, $v = v_c$ and $\dot{\varphi} = k\alpha$

Open-loop Controller

- Also, $\beta = \varphi + \alpha$ and $\dot{\beta} = \dot{\varphi} + \dot{\alpha}$ ☞ $\dot{\beta} = \omega + \dot{\alpha}$
- If the robot is far from the goal, then $\dot{\beta}$ =0 ☞ $-\omega = \dot{\alpha}$
- We have $\dot{\alpha} = -\omega = -k\alpha$
- Integrating for α

$$\alpha(t) = \alpha(0)e^{-kt}$$

- The goal is to make $\alpha(t) \to 0$



- If the value of k (gain) is large then the robot has a fast response, reaches the goal quickly, but sweeps past the goal and oscillates about the goal point.
- If the value of k (gain) is small then the robot has a slow response, reaches the goal slowly, better control.
- How do we fix this? Feedback!

Feedback Proportional Control – V1

- Control input is proportional to the error, $e(t)$

$$u(t) = K_p e(t)$$

- Assume you have two points $t_0: (x_0, y_0)$ and $t_1: (x_1, y_1)$, we define two error terms e_1 and e_2

$$e_1(t) = \sqrt{(x_1 - x_0)^2 + (y_1 - y_0)^2}$$

$$e_2(t) = \tan^{-1}\left(\frac{y_1 - y_0}{x_1 - x_0}\right) - \theta_0$$

- These errors can be turned directly into robot's speeds, for example using a simple proportional controller with gains k_1,and k_2 :

$$\dot{v} = k_1 e_1(t)$$

$$\dot{\theta} = k_2 e_2(t)$$

- which will let the robot drive in a curve until it reaches the desired pose

Feedback Proportional Control – V2

- Assume you have two points start:= t_0: (x_0, y_0, θ_0) and Goal:= t_1: (x_1, y_1, θ_1), we define three error terms e_1, e_2, and e_3

$$e_1(t) = \sqrt{(x_1 - x_0)^2 + (y_1 - y_0)^2}$$

$$e_2(t) = \tan^{-1}\left(\frac{y_1 - y_0}{x_1 - x_0}\right) - \theta_0$$

$$e_3(t) = \theta_1 - \theta_0$$

- These errors can be turned directly into robot's speeds, for example using a simple proportional controller with gains k_1, k_2 and k_3:

$$\dot{v} = k_1 e_1(t)$$

$$\dot{\theta} = k_2 e_2(t) + k_3 e_3(t)$$

- which will let the robot drive in a curve until it reaches the desired pose

Feedback Proportional Control – Inner Loop

- Do proportional control on the orientation of the vehicle by proportionally adjusting the differences in the wheel velocities.

$$\dot{\varphi_R} = v + k_1 e_2(t)\,,\ \dot{\varphi_L} = v - k_1 e_2(t)$$

- move at a fixed speed until it gets close to goal and then can ramp down the speed

$$\dot{\varphi_R} = k_2 e_1(t)(v + k_1 e_2(t))\,,\ \dot{\varphi_L} = k_2 e_1(t)(v - k_1 e_2(t))$$

Other forms of Control

- Feedback linearization
- Non-linear Controls

Turtlesim Demos

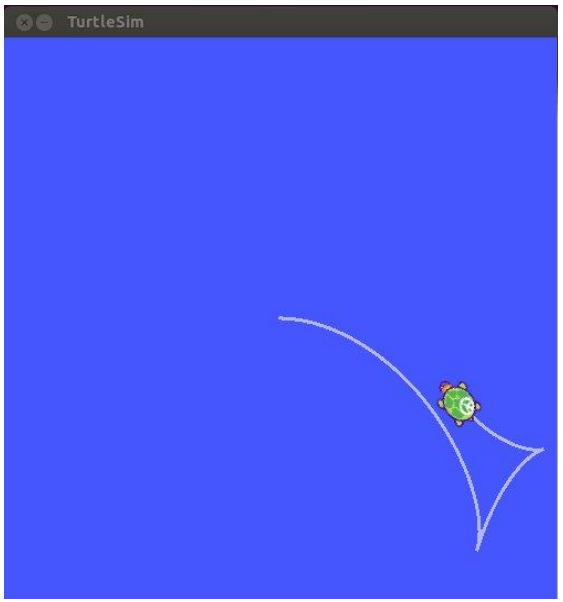
- Proportional Controller
  - https://github.com/venki666/CpE476_demos/blob/master/2022/turtlesim_cleaner/src/robot_cleaner_move_to_goal.cpp

DA 3 – PID Control

```
#include <ros/ros.h>
#include <geometry_msgs/Twist.h>
#include <geometry_msgs/Pose.h>


void PoseCallback(const geometry_msg::Pose& pose){
        current_pose = pose;
    }


int main(int argc, char **argv){
ros::init(argc, argv, "PID_Control");
ros::NodeHandle nh;ros::Subscriber pose =
nh.subscribe<geometry_msgs::Pose>("pose",1000,PoseCallback);
ros::Publisher velocity_publisher = nh.advertise<geometry_msgs::Twist>("cmd_vel", 1000);
...
}


float getDistance()
{
 return sqrt(pow((current_pose.x - gpose.x), 2) + pow((current_pose.y - gpose.y), 2));
}


float getAngle()
{
 return ((atan2(gpose.y - current_pose.y, gpose.x - current_pose.x)) - current_pose.theta);
}
```


```
void moveGoal(geometry_msgs::Pose goal_pose, double distance_tolerance)
{
//We implement a PID Controller. We need to go from (x,y) to (x',y').
geometry_msgs::Twist vel_msg;
float e_theta = 0;
float integral_theta = 0;
float diff_theta = 0;
float e_dist = 0;
float integral_dist = 0;
float diff_dist = 0;
ros::Rate loop_rate(10);
e_theta = getAngle();
e_dist = getDistance();

…
```


```
...
float k1 = 7.5, k2 = 0.00001, k3 = 0.01;
do {
 //linear velocity
vel_msg.linear.x = k1 *e_dist / 20.0) + (k2 *integral_dist) + (k3 *diff_dist);
vel_msg.linear.y = 0;
vel_msg.linear.z = 0;
//angular velocity
vel_msg.angular.x = 0;
vel_msg.angular.y = 0;
vel_msg.angular.z = (k1 *e_theta) + (k2 *integral_theta) + (k3 *diff_theta);
e_theta = getAngle();
e_dist = getDistance(); integral_theta += e_theta;
integral_dist += e_dist;

diff_dist = e_dist - diff_dist;
diff_theta = e_theta - diff_theta;

velocity_publisher.publish(vel_msg);
ros::spinOnce();
loop_rate.sleep();
} while (e_dist > distance_tolerance);
cout << "end move goal" << endl;
vel_msg.linear.x = 0;
vel_msg.angular.z = 0;
velocity_publisher.publish(vel_msg);

}
```



Using ros_controllers

- ros_controllers and associated packages
  - ***ros_controllers***: *ackermann_steering_controller* | *diff_drive_controller* | *effort_controllers* | *force_torque_sensor_controller* | *forward_command_controller* | *gripper_action_controller* | *imu_sensor_controller* | *joint_state_controller* | *joint_trajectory_controller* | *position_controllers* | *velocity_controllers*

  - *diff_drive_controller*
    - Simulation: a ROS package used to control differential drive robots, such as those with two wheels that can be driven independently. The controller allows you to command the robot's linear and angular velocities, which are then translated into wheel velocities.
    - Hardware: interfaces with the robot's hardware to send velocity commands and receive feedback. You can integrate PID control by tuning parameters to control the robot's wheel velocities and ensure smooth motion.

diff_drive_controller

- a ROS package used to control differential drive robots, such as those with two wheels that can be driven independently.
- Setup the robots URDF

```
<transmission name="left_wheel_trans">
  <type>transmission_interface/SimpleTransmission</type>
  <joint name="left_wheel_joint">
    <hardwareInterface>hardware_interface/VelocityJointInterface</hardwareInterface>
  </joint>
  <actuator name="left_wheel_motor">
    <mechanicalReduction>1</mechanicalReduction>
  </actuator>
</transmission>

<transmission name="right_wheel_trans">
  <type>transmission_interface/SimpleTransmission</type>
  <joint name="right_wheel_joint">
    <hardwareInterface>hardware_interface/VelocityJointInterface</hardwareInterface>
  </joint>
  <actuator name="right_wheel_motor">
    <mechanicalReduction>1</mechanicalReduction>
  </actuator>
</transmission>
```


- Configure the controller (controllers.yaml)

```yaml
diff_drive_controller:
  type: "diff_drive_controller/DiffDriveController"
  left_wheel: "left_wheel_joint"
  right_wheel: "right_wheel_joint"
  publish_rate: 50
  velocity_rolling_window_size: 2
  enable_odom_tf: true
  pose_covariance_diagonal: [0.001, 0.001, 0.0, 0.0, 0.0, 0.03]
  twist_covariance_diagonal: [0.001, 0.001, 0.0, 0.0, 0.0, 0.03]
  wheel_separation: 0.5       # Distance between wheels
  wheel_radius: 0.1           # Radius of wheels
  cmd_vel_timeout: 0.25
  publish_cmd: true
```

```xml
<launch>
  <param name="robot_description" command="$(find xacro)/xacro.py '$(find your_robot_description)/urdf/robot.xacro'"/>

  <!-- Load the controller -->
  <node name="controller_spawner" pkg="controller_manager" type="spawner" respawn="false" output="screen"
    args="diff_drive_controller" />

  <!-- Start robot state publisher -->
  <node name="robot_state_publisher" pkg="robot_state_publisher" type="robot_state_publisher" output="screen">
    <param name="publish_frequency" value="50"/>
  </node>
</launch>
```


With PID

```
<transmission name="left_wheel_trans">
  <type>transmission_interface/SimpleTransmission</type>
  <joint name="left_wheel_joint">
    <hardwareInterface>hardware_interface/VelocityJointInterface</hardwareInterface>
  </joint>
  <actuator name="left_wheel_motor">
    <hardwareInterface>hardware_interface/VelocityJointInterface</hardwareInterface>
  </actuator>
</transmission>

<transmission name="right_wheel_trans">
  <type>transmission_interface/SimpleTransmission</type>
  <joint name="right_wheel_joint">
    <hardwareInterface>hardware_interface/VelocityJointInterface</hardwareInterface>
  </joint>
  <actuator name="right_wheel_motor">
    <hardwareInterface>hardware_interface/VelocityJointInterface</hardwareInterface>
  </actuator>
</transmission>
```

define the wheel joints, transmission, and actuators to work with the controller.

Create a YAML file (`controller.yaml`) to configure the `diff_drive_controller` and set PID parameters for the wheels.

```
diff_drive_controller:
  type: "diff_drive_controller/DiffDriveController"
  left_wheel: "left_wheel_joint"
  right_wheel: "right_wheel_joint"
  publish_rate: 50
  pose_covariance_diagonal: [0.001, 0.001, 0.001, 0.0, 0.0, 0.001]
  twist_covariance_diagonal: [0.001, 0.001, 0.001, 0.0, 0.0, 0.001]
  cmd_vel_timeout: 0.5
  enable_odom_tf: true

  # PID settings
  pid_gains:
    left_wheel:
      p: 100.0
      i: 0.1
      d: 0.01
    right_wheel:
      p: 100.0
      i: 0.1
      d: 0.01
```


Feedback of velocity/pose

- we can get the desired /cmd_vel and use robot kinematics to determine the motion for dt to publish odometry (topic - /odom, type - nav_msgs::Odometry). Also need TF

```
current_time = ros::Time::now();
last_time = ros::Time::now();

ros::Rate r(50); // 50 Hz
while (ros::ok()) {
    current_time = ros::Time::now();

    // Calculate the change in encoder values
    double delta_left = left_wheel_ticks - prev_left_wheel_ticks;
    double delta_right = right_wheel_ticks - prev_right_wheel_ticks;

    // Calculate wheel velocities (m/s)
    double left_wheel_vel = (delta_left / ticks_per_rev) * (2 * M_PI * wheel_radius) / (current_time - last_time).toSec();
    double right_wheel_vel = (delta_right / ticks_per_rev) * (2 * M_PI * wheel_radius) / (current_time - last_time).toSec();

    // Compute the velocity of the robot
    vx = (left_wheel_vel + right_wheel_vel) / 2.0;
    vth = (right_wheel_vel - left_wheel_vel) / wheel_base;

    // Update robot's position
    double dt = (current_time - last_time).toSec();
    double delta_x = vx * cos(theta) * dt;
    double delta_y = vx * sin(theta) * dt;
    double delta_theta = vth * dt;

    x += delta_x;
    y += delta_y;
    theta += delta_theta;
```

$$\text{Wheel Velocity} = \frac{\Delta\text{Encoder}}{\Delta t} \times \frac{2\pi}{\text{Ticks Per Revolution}} \times \text{Wheel Radius}$$

$$v = \frac{v_{\text{left}} + v_{\text{right}}}{2}$$

$$\omega = \frac{v_{\text{right}} - v_{\text{left}}}{\text{Wheel Separation}}$$

$$\Delta x = v\cos(\theta)\Delta t$$

$$\Delta y = v\sin(\theta)\Delta t$$

$$\Delta\theta = \omega\Delta t$$

Getting encoder ticks

Subscribe to encoder ticks

```
double wheel_radius = 0.1;  // in meters
double wheel_base = 0.5;    // distance between the wheels (in meters)
int ticks_per_rev = 500;    // encoder ticks per wheel revolution

double left_wheel_ticks = 0, right_wheel_ticks = 0;
double prev_left_wheel_ticks = 0, prev_right_wheel_ticks = 0;

ros::Time current_time, last_time;

void encoderCallback(const EncoderMsg& msg) {
    left_wheel_ticks = msg.left_ticks;
    right_wheel_ticks = msg.right_ticks;
}

int main(int argc, char** argv) {
    ros::init(argc, argv, "odometry_publisher");
    ros::NodeHandle nh;

    ros::Publisher odom_pub = nh.advertise<nav_msgs::Odometry>("odom", 50);
    tf::TransformBroadcaster odom_broadcaster;

    ros::Subscriber encoder_sub = nh.subscribe("encoder_topic", 10, encoderCallback);
```


Publish TF & odom

```
// Create quaternion from yaw
geometry_msgs::Quaternion odom_quat = \
       tf::createQuaternionMsgFromYaw(theta);

// Publish the transform over TF
geometry_msgs::TransformStamped odom_trans;
odom_trans.header.stamp = current_time;
odom_trans.header.frame_id = "odom";
odom_trans.child_frame_id = "base_link";

odom_trans.transform.translation.x = x;
odom_trans.transform.translation.y = y;
odom_trans.transform.translation.z = 0.0;
odom_trans.transform.rotation = odom_quat;

// Send the transform
odom_broadcaster.sendTransform(odom_trans);
```

```
// Publish odometry message
nav_msgs::Odometry odom;
odom.header.stamp = current_time;
odom.header.frame_id = "odom";

// Set position
odom.pose.pose.position.x = x;
odom.pose.pose.position.y = y;
odom.pose.pose.position.z = 0.0;
odom.pose.pose.orientation = odom_quat;

// Set velocity
odom.child_frame_id = "base_link";
odom.twist.twist.linear.x = vx;
odom.twist.twist.linear.y = 0.0;
odom.twist.twist.angular.z = vth;

// Publish odometry
odom_pub.publish(odom);

prev_left_wheel_ticks = left_wheel_ticks;
prev_right_wheel_ticks = right_wheel_ticks;
last_time = current_time;

ros::spinOnce();
r.sleep();
```


- *velocity_controllers*
  - `velocity_controllers/JointVelocityController` package in ROS is used to control the velocity of a joint by applying a PID controller.

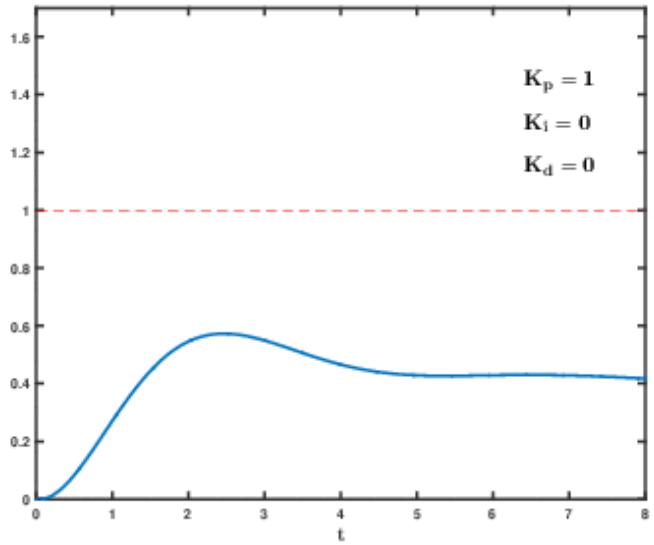
```
joint_velocity_controller:
  type: "velocity_controllers/JointVelocityController"
  joint: "wheel_joint"
  pid:
    p: 100.0
    i: 0.01
    d: 0.1
    i_clamp_min: -1.0
    i_clamp_max: 1.0
  command_timeout: 0.5  # Optional, stop after 0.5 seconds if no command is received
```

```
<launch>
  <rosparam file="$(find your_robot)/config/controller.yaml" command="load"/>

  <node name="controller_spawner" pkg="controller_manager" type="spawner" args="joint_velocity_controller" />
</launch>
```

In all cases you need to provide /cmd_vel

Steps to Fine-Tune PID Values

- **Start with Zero I and D Gains**:
  - Begin by setting the **I (Integral)** and **D (Derivative)** terms to 0.
  - Adjust the **P (Proportional)** gain first until you observe oscillations. The goal is to find the point where the system becomes responsive but does not oscillate excessively.
- **Tune the P Gain (Proportional)**:
- **Tune the I Gain (Integral)**:
- **Tune the D Gain (Derivative)**:
- **Test Under No Load and Load Conditions**:
- **Iterate**:



Related Yahboom materials

- Reference
  - http://www.yahboom.net/study/ROSMASTER-X1

- Hardware course
  - 15. PID control robot movement

- Robot control course
  - 1.PID algorithm theory
  - 2.Robot PID debugging

Assignment

- Setup your robot to be controlled by the joy-stick controller
  - Setup your robot
  - Update or reload the original firmware in the ros controller
  - Install the provided SD card image
  - Setup joy-stick ros package
- Provide /cmd_vel command to move the robot in a straight line
  - Montior the ticks, odom and visualize the topics/messages.
- Tune PID values for smooth operation.